

Skill-Portfolio

대표 기술

임베디드, 펌웨어, 디바이스

드라이버 개발

개인 정보

성명 : 박지현

생년월일 : 2000.01.04.

연락처 : 010-9502-5535

E-mail : nice121223@gmail.com

최종 학력사항

재학기간	학교명 및 전공
2024.03~2027.02	상명대학교 전자공학과

교육 이수사항

교육명	내용	교육기관
AI시스템 반도체 SW개발자	- C/C++ 및 RTOS 기반 실시간 제어 펌웨어 설계 - 임베디드 리눅스 환경의 디바이스 드라이버 구현 - Verilog HDL 기반 RTL 설계 및 FPGA 시스템 검증	대한상공회의소 서울기술교육센터
반도체 공정 실습 교육	산화·막증착·포토·식각 등 주요 반도체 공정 이론 및 프로세스 학습	한국기술교육대학교

	• 공정 시뮬레이션 실습을 통한 Diode·MOSFET 소자 제조 공정 이해 • Class 1,000 클린룸 환경에서 막형성·포토식각 장비 활용 실무 실습	
--	---	--

자격 사항		
자격증명	취득일	시행기관
OPlc(IM1)	2025.11.15	ACTFL

전문설계 소프트웨어 능력	
언어(툴)	활용능력
C/C++ (STM32CubeIDE, PlatformIO, GCC, Linux)	ARM Cortex-M 기반 MCU Peripheral 제어(UART·SPI·I2C), 포인터 활용 Driver 설계, RTOS 태스크 스케줄링, 임베디드 리눅스 캐릭터 디바이스 드라이버 구현(app/dev 분리)
Verilog (Vivado, Quartus, VCS Verdi, Design Compiler)	FSM 기반 RTL 설계, 통신 프로토콜(RTC·UART) 구현, 테스트벤치·오실로스코프 기반 HW 검증
Python (Anaconda, Jupyter Notebook)	CNN 기반 이미지 세그멘테이션 모델 학습 및 Jetson Nano 실시간 추론

프로젝트 수행사항			
번호	프로젝트명	프로젝트 내용	수행기관
1	(Team) 3축 능동형 스마트 짐벌 및 객체 추적 시스템	Dual-MCU + Edge Vision 기반 3축 자세 안정화 및 실시간 객체 추적 시스템	대한상공회의소 서울기술교육센터
2	(Team) Smart Medicine Dispenser	STM32·Arduino·RaspberryPi·MariaDB 기반 IoT 스마트 약품 자동 배출 시스템	대한상공회의소 서울기술교육센터
3	(Team) Basys3 기반스마트 공조기	DHT11·DS1302 연동 온습도 모니터링 및 RTC·알람 제어 공조 시스템	대한상공회의소 서울기술교육센터
4	(Team) U-Net 기반 누수 감지 안전 시스템	Jetson·STM32·RaspberryPi·MariaDB 온디바이스 AI 누출 감지 및 안전 제어 시스템	대한상공회의소 서울기술교육센터

프로젝트 기술서

작 성 자
박지현

1	프로젝트명 : 3축 능동형 스마트 짐벌
수행기간	2026.04.22.~2026.05.10
담당역할	• ESP32-CAM(Vision Edge) 펌웨어 설계 및 구현 • RGB565 → HSV 기반 빨강 객체 추적 알고리즘 설계 • RLE 군집화·칼만 필터 좌표 안정화 및 Board A 연동 SPI 패킷 설계
수행목표	• 제한 자원(SRAM 520KB / 스택 8KB) 내 QQVGA 20fps 실시간 추적 확보 • 조명 변화에 강건한 색상 판별 로직 구현 • 측정 노이즈 억제로 서보 진동 방지 및 안정적 SPI 스트리밍
사용 기술	[개발환경] • PlatformIO (ESP32-CAM + MB 실드를 단일 보드로 구성) • Serial Monitor 디버깅 [사용기술] • C/C++ (color_tracker.h / color_tracker.c 모듈화 → main.cpp 호출) • OV2640 카메라 제어, RGB565 · HSV · RLE · Kalman Filter • 하드웨어 SPI(Master) 패킷 통신
세부수행내용	

1. 개요

모터 제어용 MCU(STM32)의 연산 부담을 원천 차단하기 위해, 영상 캡처부터 좌표 산출까지의 비전 처리를 독립된 엣지 모듈(ESP32-CAM)에 전담

ESP32-CAM에서 빨간색 객체를 추적하여, 원본 영상과 좌표를 SPI로 Board A에 전송 → Board A는 LCD에 영상을 렌더링하고 좌표만 Board B로 중계

- 핵심 알고리즘을 color_tracker.c로 모듈화하고, 헤더(color_tracker.h)로 인터페이스를 분리하여 main.cpp에서 호출하는 구조로 설계

처리 파이프라인: 영상 캡처(RGB565) → HSV 변환 → 빨강 판별 → RLE 군집화 → 칼만 필터 → SPI 송신

2. 설계 및 검증

■ 해상도 · 픽셀 포맷 선정 (SPI 대역폭 Trade-off)

내부 SRAM만으로는 영상 처리가 불가하여 외부 PSRAM에 프레임버퍼를 할당해 캡처·분석을 병렬화함. 해상도와 픽셀 포맷은 SPI 전송 시간과 실시간성의 트레이드오프를 기준으로 결정함.

구분	데이터량	판정
QVGA / VGA	153.6KB / 614.4KB	전송 시간 초과 / PSRAM 초과 → 불가
RGB888 (QQVGA)	57.6KB	전송 지연, 실시간성 붕괴 ✕
RGB565 (QQVGA)	38.4KB	전송 15ms · 20fps 달성 ✓

■ HSV 색상 판별 및 정수 최적화

RGB는 조명 변화에 값이 크게 흔들려 고정 임계값 판별 불가 → 색상(H)/밝기(V)를 분리한 HSV 채택

빨강은 색상환 양 끝에 위치하므로 두 구간을 OR로 동시 체크 ($H \leq 10 \parallel H \geq 170$)

0~360° 각도를 절반(0~179)으로 압축하여 uint8_t(최대 255) 1바이트에 저장 → 메모리 절감
매 프레임 19,200픽셀을 float로 연산 시 코어 점유율 급증 → 백그라운드 WiFi와 충돌. 판별
범위가 넓어 소수점 정밀도가 불필요하므로 정수 비례식으로 대체

■ RLE 군집화 및 칼만 필터

재귀 DFS/BFS는 스택(8KB) 초과로 즉시 크래시 → 메모리 복잡도 $O(1)$ 의 1D 라인 스캔(RLE)
채택

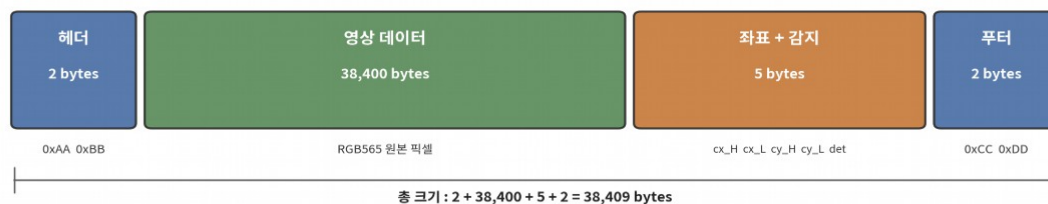
연속 2픽셀 이상($\text{run} \geq 2$)만 유효 처리 → 고립 노이즈 픽셀 자동 필터링

행마다 분리되는 런을 중심 X좌표 30px 이내면 하나의 Blob으로 병합, 면적 가중 평균으로
무게중심 갱신 (160px 폭 기준 물체 폭 20~50px → 30px 최적)

산출된 좌표는 측정 노이즈($\pm 5\text{px}$)를 서버에 직접 전달하면 진동이 발생하므로, 예측값과 측정값의
신뢰도를 융합하는 1차원 칼만 필터로 안정화함.

■ SPI 통신 패킷 설계 (ESP32-CAM → Board A)

SPI 통신 패킷 구조 (ESP32-CAM → Board A)



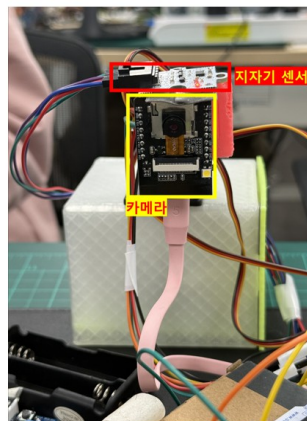
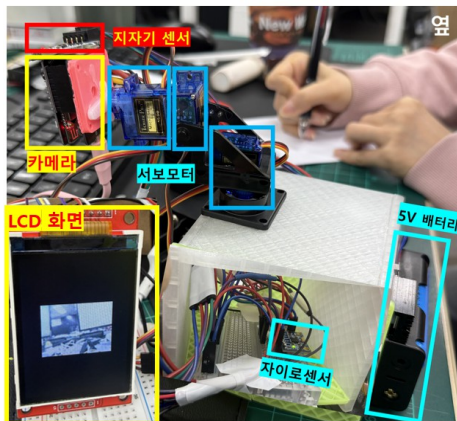
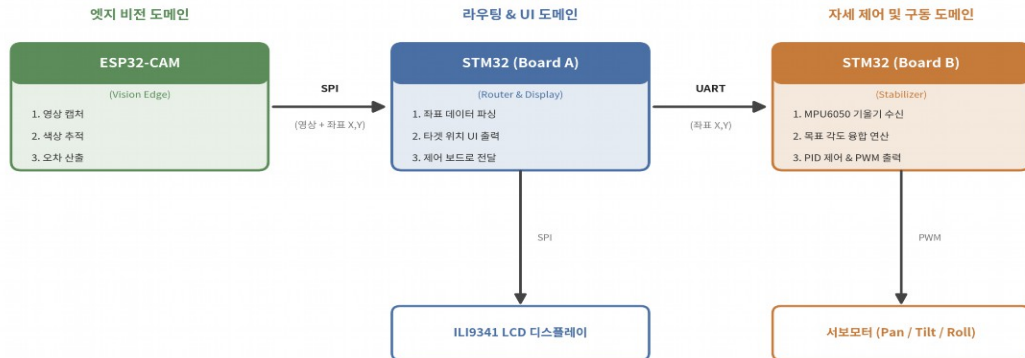
헤더/푸터 동기화: 기동 시 보드 간 타이밍 불일치로 패킷 중간부터 읽으면 영상·좌표가 섞임 →
매직 넘버를 매 프레임 검증, 깨진 패킷은 폐기 후 다음 프레임 자동 재동기화 (RGB565 영상에
해당 바이트열 연속 출현 확률 낮아 마커로 적합)

16bit 좌표 바이트 분할: SPI 버스를 8bit 모드로 통일했기에 16bit 좌표를 상/하위 바이트로 분할
전송 (최대 65,535까지 표현 → 향후 해상도 상향 시 패킷 구조 변경 불필요)

■ 하드웨어 구성 사진

3축 능동형 스마트 짐벌 및 객체 추적 시스템

분산 처리 아키텍처 (Dual-MCU + Edge Vision)



3. 고찰

■ 배운 점

비전 처리를 엣지로 물리적으로 분리하여, 무거운 영상 연산에도 모터 제어 주기가 밀리지 않는 분산 처리 아키텍처의 효과를 확인함

float→int 변환, RLE 채택 등 제한된 임베디드 자원(SRAM·스택·SPI 대역폭)에 맞춰 알고리즘을 최적화하는 과정에서, 이론적 정확도보다 실시간성·자원 효율의 Trade-off를 고려하는 설계 관점을 익힘

헤더/푸터 동기화 패킷을 직접 설계하며, 물리적으로 분리된 두 보드 간 통신에서 타이밍 불일치를 소프트웨어적으로 복구하는 방법을 배움

프로젝트 기술서

작 성 자

박지현

2	프로젝트명 : 스마트 알약 디스펜서
수행기간	2026.05.29~2026.06.03
담당역할	• Arduino 인증 단말(RFID + 4×4 키패드) 설계 및 구현 • 라즈베리파이 서버 DOC_client(처방 판단·배출 제어 FSM) 설계 및 구현 • LOG_client(회원가입·로그인 인증) 설계 및 구현 • SQL_client 및 DB 스키마 공동 설계
수행목표	• RFID·키패드로 UserID를 입력받는 무인 인증 단말 구축 • 서버에서 다중 처방 조건(알려지·나이·24h·누적)을 자동 검증하는 로직 구현 • 서버 오류 시에도 DB 무결성을 보장하는 Fail-Safe 배출 흐름 설계 • 하드웨어(배출부)와 외부망을 분리하는 보안 구조 확립
사용 기술	[개발환경] • Arduino IDE (인증 단말) / GCC · pthreads (서버 클라이언트) • Raspberry Pi OS (Linux) / MariaDB [사용기술] • C++ (Arduino) — MFRC522(RFID) · 4×4 키패드 · WiFiEsp(ESP-01) • C언어 (서버) — TCP 소켓 통신, doc_client / log_client / sql_client • SQL — SmartDispenser DB (UserTable / MedicineRule / Inventory 등)
세부수행내용	

1. 개요

사용자가 RFID 카드 또는 키패드로 UserID를 입력하면, 라즈베리파이 IoT 서버가 DB 기반으로 처방 조건을 자동 검증하고 서보모터로 약품을 배출하는 무인 시스템에서, 사용자 인증부터 서버 처방 판단까지의 세로 흐름 담당함

배출부(STM32)와 인증 단말(Arduino)은 물리적으로 직접 연결하지 않음. 모든 명령은 오직 라즈베리파이 서버를 경유하며, doc_client가 판단·중계함으로써 외부망과 하드웨어 제어부를 분리하여 물리 보안을 확보함

처방 규칙을 코드가 아닌 DB 테이블로 관리하여, 재컴파일 없이 약품·나이·알러지·재고 조건을 변경할 수 있도록 설계함

2. 설계 및 검증

■ 시스템 아키텍처 및 보안 구조 (클라이언트 권한 분리)

TCP 기반 IoT Server를 중심으로 각 클라이언트를 연결하는 멀티클라이언트 구조를 설계함. 서버는 메시지 발신자 ID를 수신자 ID로 교체해 포워딩하며, 클라이언트별 접근 권한을 제한함

클라이언트	허용	제한
AND(스마트폰 앱)	SYMPTOM / LOGIN / CREATE 요청	SQL 직접 접근 불가
ARD(입력 단말)	SYMPTOM / PICKUP 요청	STM 직접 명령 불가
STM(배출기)	PICKUP_DONE / FAIL 보고	코드 직접 수정 불가
DOC(처방 판단)	처방 로직 수행 및 데이터 업데이트 권한 (SQL에 요청)	DB 쿼리 직접 실행 불가 (반드시 SQL 경유)
LOG(회원가입/로그인)	사용자 계정 검증 및 인증 로직 처리	처방 데이터에 접근 불가, DB 쿼리 직접 실행 불가
SQL(DB 게이트 웨이)	CRUD 처리 (GET/SET/UPDATE)	처방 판단 로직 없음

■ 처방 판단 및 Fail-Safe 배출 (DOC_client)

단말이 [DOC]SYMPTOM@UserID@증상을 전송하면, doc_client가 ① 알려지 → ② 24h 한도 → ③ 누적 복용량 → ④ OVERDOSE → ⑤ 나이 → ⑥ 중복 예약 순으로 조건을 검증하여, 하나라도 위배되면 해당 FAIL 코드를 반환하고 처방을 차단함. 이 다중 검증으로 청소년 약물 오남용과 과용량을 서버 단에서 원천 차단함.

약 배출 시에는 상태 변수(g_step)로 순차 처리하되, DB 변경은 STM32의 물리적 배출 완료(PICKUP_DONE) 신호를 수신한 이후에만 수행하도록 설계함. 서버 오류로 PICKUP_FAIL이 수신되면 DB를 변경하지 않고 유지하여 재시도가 가능하며, 배출이 실제로 확인되지 않으면 재고·누적·예약 코드가 그대로 보존되어 무결성이 유지됨.

UUUUUU: SmartDispenser, UUUU: DispenseLog, UU: UU UUUU

LogID	UserID	MedicineCode	DispenseTime
6	123456	1	2026-05-30 14:50:43
10	10303	103	2026-06-01 09:40:23
11	10303	210	2026-06-01 09:41:24
12	10303	415	2026-06-01 09:43:33
13	10303	303	2026-06-01 09:44:37
14	10303	103	2026-06-01 09:45:17
15	123456	101	2026-06-01 10:58:46
16	10303	101	2026-06-01 11:18:12
17	10303	101	2026-06-01 13:29:07
18	10303	101	2026-06-01 13:31:26
19	10303	101	2026-06-01 13:32:21
20	10303	101	2026-06-01 13:38:53
21	10303	101	2026-06-01 13:39:26
22	10303	101	2026-06-01 13:40:09
23	10303	101	2026-06-01 13:41:01
24	10303	101	2026-06-01 13:41:44
25	10303	101	2026-06-01 13:42:01
26	10303	103	2026-06-01 13:42:31
27	10303	103	2026-06-01 13:44:33
28	10303	103	2026-06-01 13:45:46
29	10303	103	2026-06-01 13:46:33
30	10303	103	2026-06-01 13:46:58
31	10303	103	2026-06-01 13:48:46
32	10303	103	2026-06-01 13:50:25
33	10303	201	2026-06-01 13:53:34
34	10303	301	2026-06-01 13:54:03
35	10303	401	2026-06-01 13:54:54
36	10303	501	2026-06-01 13:55:31
37	10303	201	2026-06-01 14:50:03
38	10303	201	2026-06-01 14:50:39
39	10303	203	2026-06-01 14:53:25
116	11111	102	2026-06-02 17:00:31
117	11111	102	2026-06-02 17:02:20
118	22222	101	2026-06-02 17:05:23
119	22222	101	2026-06-02 17:06:56
120	77777	102	2026-06-02 17:10:20
121	66666	101	2026-06-02 17:14:34
122	66666	101	2026-06-02 17:15:40

UserID	UserName	UserAge	Allergy	MedicineCode	CumMedicine
11111	NormalUser	28	NONE	0	4
12345	CMJ	15	NULL	101	0
22222	TeenUser	16	NONE	0	2
33333	AllergyUser	35	NSAID	0	0
44444	WarnUser	29	NONE	101	12
55555	OdUser	42	NONE	0	22
66666	YoungUser	17	NONE	0	2
70707	jh	30	NONE	0	0
77777	ResvUser	33	NONE	0	2
80808	lh	30	NONE	102	0

[Dispenslog]

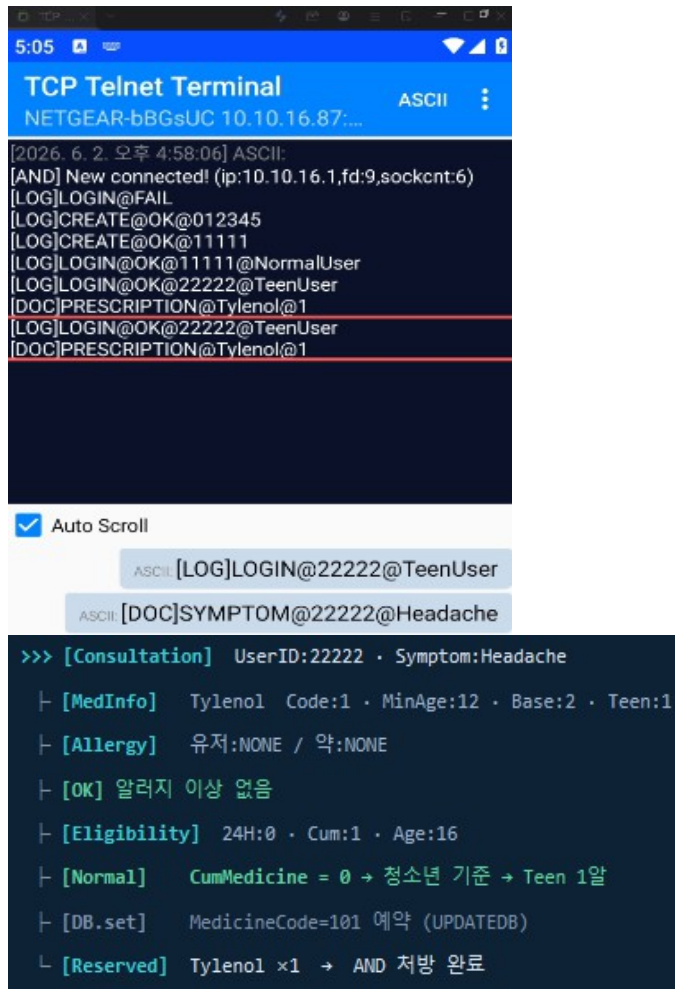
[UserTable]

SlotNum	MedicineCode	Stock	Ingredients	MedName	ServoChannel
1	1	40	Acetaminophen 500mg	Tylenol	1
2	2	95	Digestive Enzyme	Digestant	2
3	3	99	Butylscopolamine 10mg	Buscopan	3
4	4	100	Ibuprofen 200mg	Ibuprofen	2
5	5	100	Diphenhydramine 25ma	SleepAid	3

[Inventory]

■ Arduino 인증 단말 구현

MFRC522로 카드 UID를 읽어 UserID에 매핑하고, 기존 예약이 있으면 즉시 배출을, 없으면 키패드로 증상을 선택받아 서버에 전송함. 다중 클라이언트를 경유하는 통신 특성상 응답 지연이 발생하므로, delay()를 제거하고 millis() 타이머 기반 비블로킹 방식으로 구현하여 WiFiEsp 버퍼 오버플로우를 방지함.



[Android 앱 실시간 처방 로그]

[doc_client 실시간 로그]

3. 고찰

■ Trouble Shooting

문제 상황	원인	해결
배출 실패에도 DB 업데이트됨	PICKUP 시도만으로도 DB가	배출 성공 시에만 PICKUP_I

	갱신되어 이미 수령한 것으로 기록됨	전송하도록 수정 → 물리적 배후 DB 갱신
Arduino Server Timeout	DB 왕복이 여러 클라이언트를 경유해 처리 시간이 단말 대기 시간을 초과	doc_client·Arduino 응답 시간을 상향해 왕복 지연 흡수

■ 배운 점

처음에는 sql_client 하나가 조회·검증·판단을 모두 처리하도록 설계하려 했으나, 한 모듈에 책임이 몰리면 복잡도가 급증하고 장애 지점 추적과 보안이 취약해짐을 인식함

역할별로 DOC(처방 판단)·LOG(인증)·SQL(DB 게이트웨이)를 분리하니 디버깅이 쉬워지고 권한 분리로 보안이 강해졌으며, 책임 분리 설계의 실질적 이점을 체감함

인증 단말과 하드웨어 제어부를 서버로 격리함으로써, 외부 입력단을 배출부로부터 분리하는 물리 보안 구조의 중요성을 배움

프로젝트 기술서

작 성 자

박지현

3	프로젝트명 : AHU(Air Handling Unit)
수행기간	2026.03~2026.03
담당역할	• DS1302(RTC) controller / logic FSM 기반 설계 및 구현 (팀 협업) • 로터리·버튼 기반 알람 설정 및 부저 제어 로직 전체 구현 • 오실로스코프 측정 및 신호 파형 분석
수행목표	• DS1302 RTC와 SPI 유사 프로토콜로 시각 read/write 구현 • UART로 수신한 시각 보정값(setrtc)을 RTC에 반영 • 로터리 인코더로 알람 시·분을 설정하고 RTC 시각 일치 시 부저 알림 • 오실로스코프로 실제 통신 파형을 검증하여 하드웨어 동작 확인
사용 기술	[개발환경] Vivado / Basys3 Board / 오실로스코프 [사용기술] • Verilog HDL (FSM 기반 RTL 설계) • DS1302 RTC 통신 프로토콜 (CE / SCLK / bidirectional data) • 로터리 인코더 위상차 방향 판별, 부저 주파수 생성
세부수행내용	

1. 개요

Basys3 보드에 DHT11(온습도)·DS1302(RTC)를 연동한 스마트 공조기에서, RTC 시각 관리와 알람 제어를 담당함

DS1302는 시각을 FND·UART로 출력하고, PC(ComPortMaster)에서 setrtc yymdddhmmss 명령으로 시각을 보정함

알람은 DS1302 모드에서 SW0 ON 시 설정 모드로 진입하여, 로터리로 시·분을 맞추고(LED0 ON) RTC 시각과 일치하면 부저로 알림, KEY로 해제함

2. 설계 및 검증

■ DS1302 RTC 통신 설계 (controller + logic 2단 FSM)

DS1302는 CE·SCLK·양방향 data 3선으로 통신하는 RTC로, 명령/데이터 처리를 controller와 logic 두 계층의 FSM으로 분리하여 설계함

controller FSM: WP(Write Protect) 해제 → 시각 6바이트(sec·min·hour·date·month·year) 순차 write → WP 재설정 흐름 관리

logic FSM: IDLE → CE_High → Send CMD → RW Data → CE_Low 순환으로 실제 비트 단위 전송 담당

DS1302가 CE 상승 후 SCLK rising edge에서 데이터를 수신하는 특성에 맞춰, falling edge마다 다음 비트를 미리 o_data에 실어 보내는 방식으로 구현

inout 포트를 io_mode에 따라 입력/출력으로 전환하여, 단일 data 선으로 read·write를 모두 처리

■ UART 시각 보정 연동

UART로 수신한 setrtc 명령을 circular queue에 저장하고, DS1302가 busy가 아닐 때 48bit 시각 데이터를 write하도록 트리거함.

DS1302가 동작 중일 때 수신 데이터를 버퍼링하여 유실을 방지하는 구조로 설계

■ 로터리·버튼 알람 제어

알람 설정은 SET_HOUR → SET_MIN → SET_DONE의 FSM으로 동작

로터리 인코더의 위상차(11→01→00→10 순서 등)를 이용해 회전 방향(CW/CCW)을 판별하여
시(0~23)·분(0~59) 증감, KEY로 상태 전환

설정 완료 시 is_armed 플래그와 LED0 세움

부저 생성: RTC 현재 시각 == 알람 시각(초=0) 순간 r_alarm_ring 세움 → 500ms 주기
pattern_cnt로 비프 패턴, tone_cnt로 목표 주파수(음계) 생성하여 토글

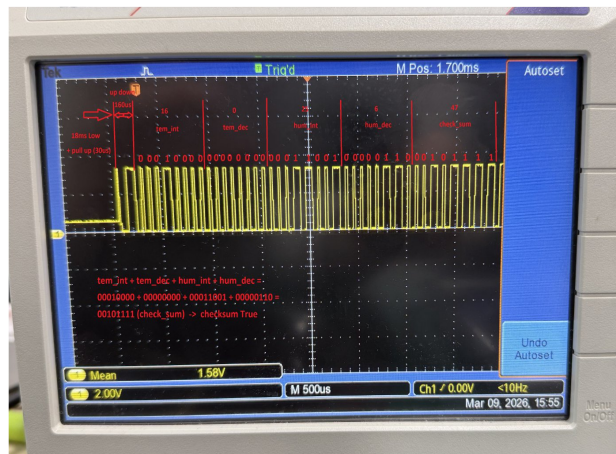
알람이 울리는 중에는 SW 상태와 무관하게 KEY로 즉시 해제되도록 하여 사용자 조작 우선순위
보장

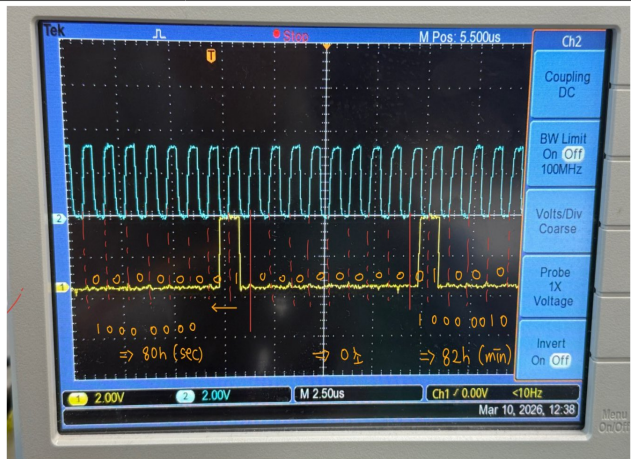
■ 오실로스코프 파형 분석

테스트벤치 시뮬레이션만으로 검증되지 않는 실제 하드웨어 동작을 확인하기 위해, DS1302의
CE·SCLK·data 신호를 오실로스코프로 측정

setrtc 260310131800 입력 시 SCLK에 동기화된 data 비트열이 규격대로 전송되는지 파형으로
검증

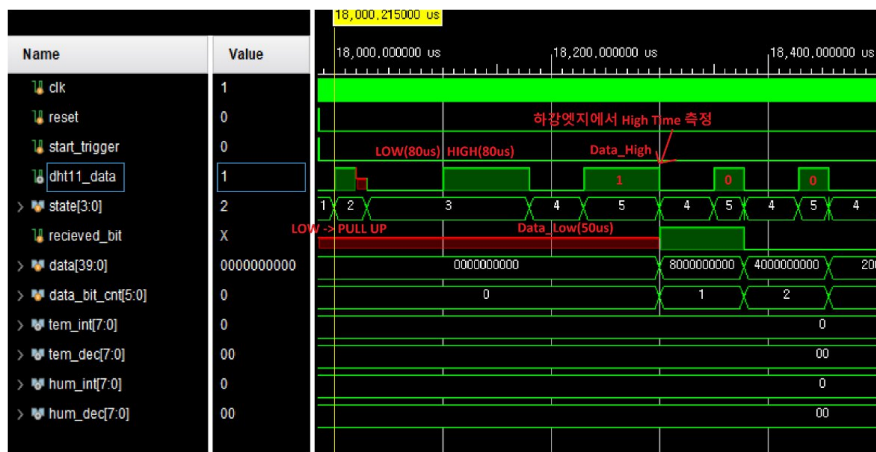
시뮬레이션 통과 이후에도 발생하는 하드웨어 이슈를 실측으로 확인



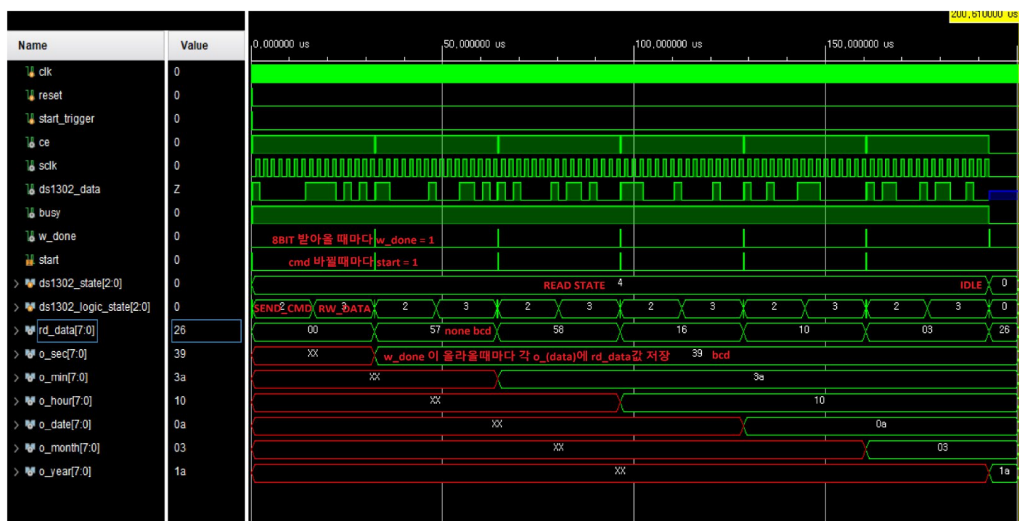


[DHT11 파형 분석]

[DS1302 파형 분석]



[DHT11 Simulation 분석]



[DS1302 Simulation 분석]

3. 고찰

■ 배운 점

프로젝트 규모가 클수록 Block Diagram과 FSM 구성을 먼저 확정하는 것이 필수적임을 체감함

— 상태 전이와 데이터 흐름을 명확히 한 뒤 구현해야 모듈 간 인터페이스 충돌이 줄어들음

디버깅 시 테스트벤치를 먼저 작성해 각 레지스터 값을 관찰하는 방식이, 코드의 어느 부분이 잘못되었는지 특정하는 데 효과적임을 배움

테스트벤치가 통과하더라도 오실로스코프 측정이 필수적임을 확인함 — 하드웨어 문제가 예상보다 빈번했고, 물리 신호를 직접 관찰해야만 잡히는 이슈가 있었음

time_out·physical issue 등 에러 발생 가능 지점을 파악했으나 시간 관계상 처리하지 못한 점이 아쉬웠으며, 향후 error controller로 에러를 모아 일괄 처리하는 개선을 구상함

프로젝트 기술서

작 성 자

박지현

4	프로젝트명 : U-Net 기반 액체 누출 감지 안전 제어 시스템
수행기간	2025.07~2026.07
담당역할	• Raspberry Pi 중계 서버 설계 및 구현 (제어·기록 경로 분리) • MariaDB 로깅 스키마(2-테이블) 설계 및 실시간 로그 도구 구현 • STM32F411 안전 제어 펌웨어 구현 (릴레이·서보·부저·LED)
수행목표	• AI 누출 판정(Jetson)과 안전 제어(STM32) 분리로 페일세이프 구조 확보 • Jetson 판정을 STM32 규약(12B·CRC16) 프레임으로 변환·중계 • 면적 임계값·통신 타임아웃 기반 7상태 FSM으로 액추에이터 제어 • 상태 시계열·FSM 전이를 분리 기록하여 사고 이력 추적
사용 기술	[개발환경] • Raspberry Pi (Linux) / STM32CubeIDE • MariaDB [사용기술] • STM32 : UART IRQ·링버퍼, CRC16-CCITT 검증, SysTick 타임아웃, GPIO/PWM 제어 • FSM 안전 제어 (7상태 · 100ms · de-energize 페일세이프) • Raspberry Pi : TCP 소켓 서버 · 스레드 분리(제어/기록) · 프레임 변환 • MariaDB : 2-테이블 스키마 · C 클라이언트 INSERT · 실시간 tail 로그
세부수행내용	

1. 개요

- Jetson Nano가 U-Net으로 색상 액체 누출을 픽셀 단위 감지하고 면적·확산·위험구역(ROI)으로 4단계 위험을 판정하면, 라즈베리파이 서버가 이를 STM32 규약으로 변환·중계하고 STM32가 최종 액추에이터를 제어하는 안전 시스템에서, Jetson 판정 중계부터 STM32 페일세이프 제어까지의 흐름을 담당함
- AI의 누출 판단(Jetson)과 안전 규칙·제어(STM32)를 물리적으로 분리하여, 상위 계통(카메라·AI·서버·통신)이 모두 정지해도 STM32가 자체 시간 기준으로 전 계통을 차단하도록 설계함
- 상태 데이터를 코드가 아닌 DB 테이블로 관리하여, 사고 이력과 상태 시계열을 분리 기록하고 사후 추적이 가능하도록 설계함

2. 설계 및 검증

■ Raspberry Pi 중계 서버 (제어/기록 경로 분리)

- Jetson으로부터 TCP(:5000)로 누출 판정({state}·{area}·{spread}·{zone})을 수신하면, 형식·상태·값 범위를 검증한 뒤 STM32 규약인 12바이트 프레임(헤더 + 상태/면적 + CRC16-CCITT)으로 변환함
- 제어 경로와 기록 경로를 분리하여 설계함. 제어 경로는 수신 루프에서 12B 프레임을 즉시 HC-06(UART)로 송신하는 블로킹 없는 단일 경로로 구성하여 지연 요소를 배제함
- 기록 경로는 별도 writer 스레드 큐를 경유해 MariaDB에 비동기 INSERT하되, 큐가 가득 차면 프레임을 드롭하고 카운트만 증가시킴. 이로써 DB·네트워크 지연이 실시간 차단 제어에 전파되지 않도록 격리함

■ MariaDB 로깅 스키마 및 실시간 로그 (2-테이블)

- 10Hz 상태 데이터를 1초로 다운샘플한 leak_sample(time·state·area%·spread·zone·comm)로 상태 시계열을 관리하여 시간대별 누수 통계를 남김
- FSM 전이 순간만 기록하는 leak_event(from→to·area%·latch·comm)를 별도 테이블로

분리하여 사고 이력과 래치 추적을 담당하게 함

- 두 테이블을 실시간 병합 출력하는 tail 도구(leak_tail)를 구현하여, 시연 중 상태 흐름(S)과 FSM 전이 이벤트(*E)를 하나의 콘솔에서 동시에 확인하도록 함

```
10.10.16.87 (t) ./leak_tail
S = leak_sample (1초 샘플)   *E = leak_event (FSM 전이)
T  time      state      area% sp zn  fsm      lat comm
-----
S  15:47:48  NORMAL      0.00 0 0  NORMAL      0    1
S  15:47:49  NORMAL      0.00 0 0  NORMAL      0    1
S  15:47:50  NORMAL      0.00 0 0  NORMAL      0    1
S  15:47:51  NORMAL      0.00 0 0  NORMAL      0    1
S  15:47:53  NORMAL      0.00 0 0  NORMAL      0    1
S  15:47:54  NORMAL      0.00 0 0  NORMAL      0    1
S  15:47:55  NORMAL      0.00 0 0  NORMAL      0    1
S  15:47:56  NORMAL      0.00 0 0  NORMAL      0    1
*E 15:47:57  NORMAL  -> CAUTION (직전 52s) area= 0.31% spread=0 zone=0 la
tch=0 comm=1
S  15:47:57  NORMAL      0.19 0 0  NORMAL      0    1
S  15:47:57  SMALL_LEAK  0.31 0 0  NORMAL      0    1
*E 15:47:58  CAUTION  -> LEAK (직전 0s) area= 0.91% spread=1 zone=0 la
tch=0 comm=1
S  15:47:58  WARNING      0.91 1 0  CAUTION      0    1
*E 15:47:59  LEAK      -> CAUTION (직전 1s) area= 1.16% spread=0 zone=0 la
tch=0 comm=1
S  15:47:59  SMALL_LEAK  1.15 0 0  LEAK          0    1
S  15:48:00  SMALL_LEAK  1.19 0 0  CAUTION      0    1
```

[MariaDB 로그]

■ STM32F411 안전 FSM 펌웨어

- UART 인터럽트로 12B 프레임의 링버퍼에 적재하고, CRC16-CCITT로 무결성을 검증하여 불일치 프레임은 즉시 폐기함. 이후 파싱(state·area)한 값을 100ms 주기 FSM에 반영함
- INIT → NORMAL → CAUTION → LEAK → DANGER의 면적 기반 5상태와, COMM_WARN·COMM_TRIP의 통신 상태 2개를 더한 7상태 FSM으로 액추에이터(릴레이·서보·부저·LED)를 제어함
- 수신 면적 비율을 임계값(0.30 / 1.50 / 3.50%·ROI)과 비교하여 상태를 승격하되, 확산(spread)은 단독 판정이 아닌 한 단계 승격 근거로만 사용하고, 센서 오류가 3초 연속되면 누출로 취급하여 "모르는 상태는 안전하게" 원칙을 적용함
- SysTick 기준으로 통신 침묵 시간을 자체 계측하여 500ms → COMM_WARN, 1.5s → COMM_TRIP으로 전이시킴. 통신이 완전히 두절되어도 STM32가 독립적으로 차단을 결정함

- 액추에이터를 de-energize 방식으로 구성하여, 통신 두절뿐 아니라 전원 상실·단선 시에도 차단 상태로 수렴하도록 설계함
- 서보모터를 릴레이의 전기적 차단과 별도의 기계적 차단 밸브로 사용하여 물리적 이중 보호 계층을 구성하고, "안전이 확실한 상태에서만 열고 그 외에는 기본 닫힘" 원칙으로 제어함

상태	면적 기준	펌프	도크 전원	표시
INIT	첫 패킷 이전	차단	차단	소등
NORMAL	< 0.30 %	통전	통전	녹색
CAUTION	≥ 0.30 %	통전	통전	황색
LEAK	≥ 1.50 %	차단	유지	황색 점멸
DANGER	≥ 3.50 % 또는 ROI(위험구역)	차단	차단	적색 + 부저
COMM_WARN	500 ms 무수신	차단	유지	황색
COMM_TRIP	1.5 s 무수신	차단	차단	적색

[안전 FSM]

■ 서보 = 기계적 차단 밸브 (이중 안전)

릴레이의 전기적 차단과 별도로 서보를 기계적 차단 밸브로 사용하여 물리적 이중 보호 계층 구성

FSM 원칙 : "안전이 확실한 상태에서만 열고, 그 외에는 기본 닫힘"

3. 고찰

■ 트러블슈팅 — 차단과 래치의 분리

문제 상황	원인	해결
손이 웹캠 앞을 스치기만 해도 즉시 DANGER·래치가 걸려 매번 보드를 리셋해야 함	즉시 차단과 래치가 같은 트리거를 공유하여, 1프레임 반응 민감도가 래치에까지 그대로 적용됨	즉시 동작(차단)과 확정(래치)을 분리 → 차단 유지, 래치·와이퍼는 위험 5초 연속 유지될 때만 확정
잡은 래치로 현장에서 래치 기능을 꺼둘 유인 발생 → 진짜 사고에도 래치 미작동 우려	오검출에도 래치가 반복 작동	스친 손은 약 0.2초 후 자 실제 위험만 5초 뒤 래치 시점은 동일하여 안전 여유

■ 배운 점

- 안전 시스템에서는 AI 판단과 안전 규칙을 물리적으로 분리하는 것이 신뢰성의 핵심임을 인식함 — 상위 계통 장애를 STM32가 독립적으로 커버하는 구조가 페일세이프의 본질임을 체감함
- 실시간 제어 경로와 기록 경로를 분리하지 않으면 DB·네트워크 지연이 그대로 제어 지연으로 전파됨을 확인하고, 경로 격리 설계의 실질적 이점을 배움
- 즉시 반응과 지연 확정을 하나의 조건으로 묶으면 오히려 안전 기능이 현장에서 무력화될 수 있음을 확인하고, 민감도와 확정을 분리 설계해야 함을 배움